AI Invoice Assistant

Architecture and technical exploration

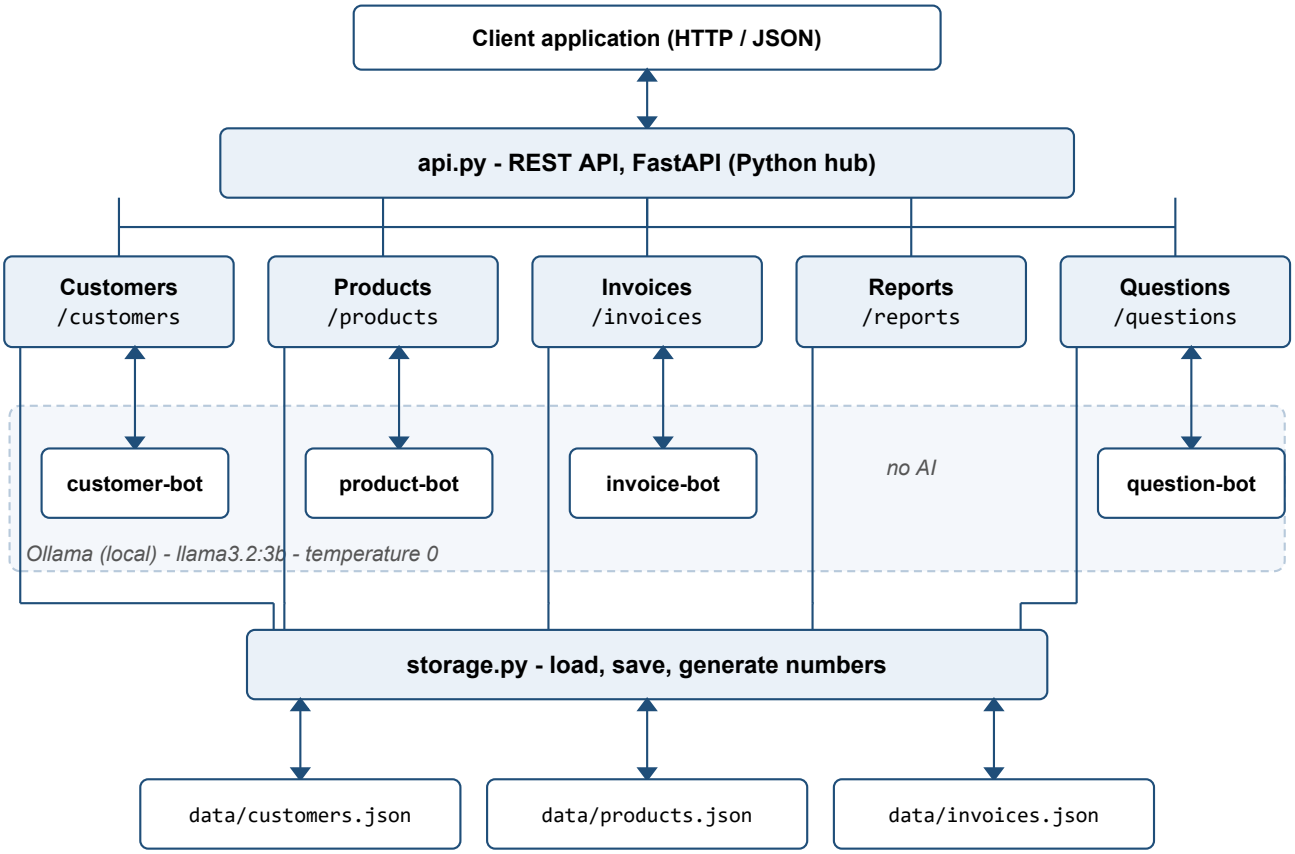
Version 1.3 - 3 October 2026

1. Service design

1.1 Options considered

Question	Options considered	Chosen	Why
How many bots?	One bot for everything / one bot per area	One per area	A 3B model follows short, focused prompts more reliably
How do the bots coordinate?	Bots call each other / Python as the hub	Python hub	Simple and predictable; a 3B model can't coordinate reliably
Who calculates?	The AI / Python	Python	Small models make arithmetic mistakes
Who confirms?	The AI asks yes/no / the API stores a pending change / the client sends the checked proposal to a CRUD endpoint	Client, after the user's yes	The bot skipped its own confirmation in tests; nothing has to be stored before the user agrees
Separation	Chat and saving in one flow / chat endpoints that only propose + plain CRUD endpoints	Chat proposes, CRUD saves	The CRUD part works and can be tested without AI, and other programs can use it
Storage	JSON files / SQLite / server database	JSON files	Small amount of data, readable, no setup
Interface	Terminal / REST API / web GUI	REST API	The final system is an API, so any client application can use it
API framework	FastAPI / Flask	FastAPI	Validates requests automatically and generates interactive documentation (`/docs`) to test every endpoint
Conversation state	Client sends the whole history each time / server keeps the conversation	Server, by `conversation_id`	The client stays simple and can't change what the bot said before
Access	Open on the network / only on this computer	Only `127.0.0.1`, no login in v1	No authentication is needed while the API can't be reached from outside
Calling Ollama	`ollama run` on the command line / REST API / `ollama` Python library	REST API with Python's `urllib`	`ollama run` packs everything into one text message (worse results in tests); `urllib` needs no extra library

1.2 Architecture



The bots never talk to each other and never touch the files. Every exchange passes through Python.

1.3 Components

Component	Role
`api.py`	FastAPI application: chat endpoints (with conversations by conversation_id) and CRUD endpoints
`customers.py`, `products.py`, `invoices.py`	Per entity: the checks (used by the chat **and** again by CRUD), the bot conversation that builds a checked proposal with a summary, and the save functions used by CRUD
`bot.py`	Sends a conversation to an Ollama bot and returns its reply
`reports.py`	R1, R2, R3: compute the turnover from the given dates or year and filters. No AI.
`questions.py`	Computes key figures and passes them to the question bot
`storage.py`	Loads and saves the JSON files, generates numbers (CUS-001, PRD-001, INV-001)
4 Ollama bots	Understand the user and ask for missing fields one at a time

1.4 Endpoints

Endpoint	Purpose	AI

`POST /customers/chat`
`POST /products/chat`
`POST /invoices/chat`
	Send one message of a create / update / delete conversation. Body: `conversation_id` (optional), `message`. Returns the bot's question, or a checked `proposal` with a `summary`. **Never saves anything.**	Yes

`POST /customers`, `PUT /customers/{id}`, `DELETE /customers/{id}` **same for** `/products` **and** `/invoices`	Create, update, delete. Checks the data again, then saves and assigns the number. The client calls these after the user said yes to the summary.	No
`GET /customers`, `GET /customers/{id}` **same for** `/products` **and** `/invoices`	Retrieve all records or one record (an invoice with its lines and total)	No
`GET /reports/turnover-by-customer`	R1. Parameters: `date1`, `date2` (DD/MM/YYYY)	No
`GET /reports/turnover-by-product`	R2. Parameters: `date1`, `date2`	No
`GET /reports/turnover-by-month`	R3. Parameters: `year`, optional `customer_id`, optional `product_id`	No
`POST /questions`	Free question about the invoices	Yes

1.5 Flow of one request (example: "delete Wilfried")

1. The client sends `POST /customers/chat` with the message "delete Wilfried".
2. Python loads the current data, adds it to the message as a `<CUSTOMER DATA>` block, and sends it to the bot together with the earlier messages of this conversation.
3. The bot replies with a question, or with one command such as `DELETE_CUSTOMER {"id": "CUS-002"}`.
4. Python checks the command: does the record exist, are any values missing, is the customer used in an invoice? If it's invalid, Python tells the bot what's wrong and the API returns the bot's new question.
5. If it's valid, the API returns the `proposal` (e.g. delete CUS-002) and the `summary` "Delete customer CUS-002: Wilfried Becker". Nothing is saved.
6. The client shows the summary and asks the user yes / no. On "no" nothing happens.
7. On "yes" the client calls `DELETE /customers/CUS-002`. Python checks again (the data may have changed in the meantime), deletes and returns the result.

1.6 Division of responsibility

AI (bots)	Python (API)	Client
Understanding natural language	Checks (in the chat and again before saving)	Shows the summary
Asking for missing fields one at a time	Numbers and codes	Asks the user yes / no
Finding the record the user means	All calculations (totals, turnover, amounts owed)	On yes: calls the CRUD endpoint
Phrasing answers	Conversation state and saving	

2. Model selection

2.1 Requirements for the model

- â€¢ It runs **locally and offline**, so no customer data leaves the computer.
- â€¢ It runs on an ordinary office PC with no dedicated GPU, with acceptable response times.
- â€¢ It follows instructions reliably: one question at a time, and an exact command format.
- â€¢ It produces valid JSON in its commands.
- â€¢ It understands names, dates and quantities in short user messages.

2.2 Candidates considered

Model	Size on disk	Strengths	Weaknesses	Decision
`llama3.2:1b`	1.3 GB	Very fast, small	Weak at following multi-step instructions and keeping a format	Rejected: too unreliable
`llama3.2:3b`	2.0 GB	Good balance of speed and instruction following; designed for on-device use	Makes mistakes in arithmetic and long conversations	**Chosen**
`qwen2.5:7b`	4.7 GB	Stronger reasoning and JSON output	More than twice as large, noticeably slower without a GPU	Rejected; kept as a fallback
Cloud models (GPT, Claude, ...)	-	Most capable	Data leaves the company, needs internet, costs per use	Rejected: offline and privacy requirement

The task description specifies `llama3.2:3b`. This comparison confirms it as a sensible choice, as long as the design makes up for its weaknesses: Python does the calculations, the validation and the confirmation (see section 1).

2.3 Parameters

Parameter	Value	Reason
`temperature`	0	The same input always gives the same output, so tests can be reproduced and the format stays stable
`num_ctx` **(context window)**	Decided during implementation	Every message carries the data block. With many records, the default context window could cut off data, so it has to be raised or the data block kept short (see risk analysis)

2.4 Customisation method

Option	Decision
Fine-tuning the model	Rejected: needs training data, time and hardware
System prompt in a Modelfile plus data in each message	**Chosen**: quick to change, and each area gets its own small prompt

3. Prototypes

The prototypes were built step by step. Each one answered one question, and its result decided the next design step. The prototype code was removed after the review. The findings are recorded here and will be applied when the application is built.

3.1 Overview

#	Prototype	Question	Result	Decision
P1	One bot for everything	Can one system prompt handle all CRUD flows and the questions?	Designed but not built: one prompt would have to cover 12+ different flows	One bot per area
P2	Customer bot v1, called with `ollama run`	Can the bot run a create/delete conversation including the yes/no confirmation?	Partly	Always send the data; never let the bot write a data block itself
P3	Same bot called through the chat API, with 2 data formats	Which message format works best?	Format B found the right customer, but skipped the confirmation	Use format B; Python asks for the confirmation
P4	Customer bot v2, with no confirmation in the prompt	Does the bot produce correct commands over several turns?	Finds the right customer, but sloppy in the flow	Python checks every command
P5	Python validation layer	Can Python catch every bot mistake seen in P2-P4?	21 out of 21 test cases passed	Keep this design

3.2 Observed bot behaviour (P2-P4)

Test	What the bot did	Problem
"add a new customer", sent without data	Asked for the name, but **invented** a data block with two fake customers (John Doe, Jane Doe)	The bot invents data
Same, with an empty data block	Asked the right question, but **repeated the data block** in its reply	Internal data visible to the user
"delete Maria", using `ollama run`	**Invented** a customer "John Doe, CUS-002" instead of finding Maria	Wrong record
"delete Maria", format A (plain text)	Asked for confirmation without showing which customer	Unclear confirmation
"delete Maria", format B (tags + JSON)	Correct ID `CUS-001`, but **deleted with no confirmation**	Dangerous
"delete Peter" (unknown customer)	Asked again, but didn't say that Peter doesn't exist	Unclear message
"change Maria's email"	Sent the command **too early**, with `"value": "..."`	Incomplete command
Creating a customer over 5 turns	Accepted an invalid email at first and only objected one question late; the final command was correct	Validation by the bot is unreliable

3.3 Lessons learned

1. The bot is good at **understanding** the user and finding the right record.
2. The bot is unreliable at **process rules**: confirmation, the order of questions, and validation.
3. So every rule that has to hold is enforced in **Python**, not in the prompt: validation, the yes/no confirmation, number generation, calculations, and removing any data block the bot repeats.
4. The data is sent through the chat API as tagged JSON (format B).

3.4 Test results of the validation layer (P5)

Function	Cases tested	Result
Reading the command (`parse_command`)	Valid command, placeholder, normal question, broken JSON, echoed data block	5 out of 5 OK
Validation (`validate`)	Valid create/update/delete, missing value, invalid value, changing the number, unknown number, customer used in an invoice	9 out of 9 OK
Confirmation text (`describe`)	Create, update, delete	3 out of 3 OK
Saving (`apply`)	Create x2, update, delete, numbers not reused, spaces removed, extra field dropped	4 out of 4 OK

4. Risk analysis

4.1 Scale

- â€¢ **Likelihood (L):** Low / Medium / High, meaning how likely the risk is to happen without any countermeasure.
- â€¢ **Impact (I):** Low / Medium / High, meaning how much damage it does to the data or to the user.
- â€¢ **Residual risk:** what's left after the countermeasure.

4.2 AI behaviour

ID	Risk	Evidence	L	I	Countermeasure	Residual
RK1	The bot invents records or numbers	Seen in P2 (John Doe, CUS-002)	H	H	The data is always sent with the message; Python checks that every number exists; numbers are generated only by Python	Low
RK2	The bot saves or deletes without confirmation	Seen in P3	H	H	The chat never saves: it only returns a proposal. Saving needs a separate CRUD call by the client after the user's yes	Low
RK3	Incomplete or invalid commands (`"..."`, invalid values)	Seen in P4	H	M	Python validation; the bot is told the error and asks again (max. 2 retries)	Low
RK4	Wrong numbers in answers to free questions	Known weakness of small models	H	H	Python calculates every figure in advance; the bot only reads them out; tested by acceptance criteria AC-Q1 to AC-Q4	Medium
RK5	The bot picks the wrong record (e.g. two customers with similar names)	Possible	M	H	The confirmation summary shows the full record before anything changes	Low
RK6	Internal data shown to the user	Seen in P2 and P4	H	L	Python removes any repeated data block before display	Low
RK7	Text inside the data (e.g. a customer name) is read by the bot as an instruction	Possible	L	M	The data is sent as tagged JSON; every command is still validated and confirmed	Low

4.3 Technical

ID	Risk	L	I	Countermeasure	Residual
RK8	The data no longer fits in the context window as the files grow	M	H	Raise `num_ctx`; send each bot only the data it needs	Medium
RK9	Slow answers on a PC without a GPU	M	M	Small model, short prompts; target of 10 seconds (AC-G7)	Medium
RK10	Ollama not running, or the model missing	M	M	The API returns a clear error message when Ollama can't be reached	Low
RK11	A JSON file is damaged (crash while writing, manual edit)	L	H	Write to a temporary file first, then replace the old file	Low
RK12	Data loss (disk failure, deletion)	L	H	Outside v1; recommend backing up the `data/` folder	Medium
RK13	A model update changes the bot's behaviour	L	M	Fixed model version in the Modelfile; re-run the scenarios after any update	Low
RK18	Unauthorised access to the API	M	H	The API listens only on `127.0.0.1`; no login needed in v1. A login is required before opening it to the network	Low
RK19	Two requests write the same JSON file at the same time	M	H	A lock around every load-change-save	Low
RK20	The client gives up (HTTP timeout) while the bot is still answering	M	M	Bot target of 10 seconds (AC-G7); clients use a timeout of at least 30 seconds	Low
RK21	Conversations are lost on restart, or pile up in memory	M	L	Conversations expire after inactivity; the client simply starts a new one	Low
RK22	A confirmed proposal arrives after the data has changed (e.g. the customer was already deleted)	L	M	The CRUD endpoints run the same checks again before saving	Low

4.4 Functional

ID	Risk	L	I	Countermeasure	Residual
RK14	Turnover is misunderstood (unpaid invoices counted, wrong date used)	M	M	Exact definition in the requirements; cross-checks AC-R3 and AC-R4	Low
RK15	Date formats get confused (DD/MM vs MM/DD)	M	M	Python accepts only DD/MM/YYYY; the confirmation shows the date written out (e.g. "3 October 2026")	Low
RK16	Deleting a customer or product that is still used in invoices	M	H	Python refuses the deletion (AC-C4, AC-P3)	Low
RK17	`llama3.2:3b` turns out not to be good enough overall	M	H	Fallback to `qwen2.5:7b`: only the model line in the Modelfiles changes	Medium

4.5 Summary

Every risk that was observed in the prototypes is reduced to **Low** by rules enforced in Python. Four risks remain at **Medium** and will be watched during implementation and testing:

- â€¢ RK4: numbers in free answers,
- â€¢ RK8: context window size,
- â€¢ RK9: speed,
- â€¢ RK17: model quality.

RK12 (data loss) is also still Medium, but it's outside v1.